Declarative Handler Topology Synthesis for Arrow Effects

Luis Emidio

2025-08-23

Declarative Handler Topology Synthesis for Arrow Effects

Abstract

Algebraic effects and handlers provide a modular mechanism for separating the declaration of computational operations from their interpretation. In Sanada's calculus of algebraic effects and handlers for arrows, handler composition is represented explicitly by nested `handle` expressions. This representation exposes an important practical question: can handler topology be specified declaratively and synthesized statically, rather than written manually as nested syntax?

We present a declarative handler-topology system layered above Sanada's arrow calculus. A declarative specification describes a finite set of handlers together with precedence constraints. A compile-time synthesis pass converts this specification into a linearly nested sequence of `handle` expressions accepted by the underlying arrow calculus. The synthesis problem is therefore formulated as finding a linear extension of a handler-precedence partial order, rather than constructing a branching runtime computation.

This distinction is important because the expressiveness limitation identified by Sanada concerns effectful command branching within the arrow semantics. Our synthesis mechanism introduces no such runtime branching: it generates a fixed handler topology before execution. We formalize the synthesis transformation, characterize its well-formedness and precedence-preservation properties, and describe an implementation strategy for arrow-effect languages that lowers declarative specifications directly to existing handler-expression AST nodes. The resulting system requires no new runtime mechanism and preserves the semantic boundary imposed by the underlying arrow calculus.

1. Introduction

Algebraic effects allow programs to describe operations independently of their interpretation. Effect handlers subsequently determine how those operations are handled. This separation provides a useful foundation for modular programming, but the composition of handlers remains an explicit part of the program structure.

In Sanada's calculus for algebraic effects and handlers for arrows, handler composition is expressed by nested `handle` constructs:

```
handle (handle P with H ) with H
```

The nesting determines handler scope and therefore potentially affects program behavior. The programmer must consequently encode the desired handler topology directly in the term.

We investigate whether this topology can instead be specified declaratively.

The problem considered in this paper is narrower than general topology synthesis. We do not attempt to infer arbitrary computational graphs, physical layouts, or dynamic effect-routing structures. Rather, we consider the specific problem of **synthesizing the static nesting topology of arrow effect handlers**.

Our central observation is that the synthesis problem is naturally expressed as an ordering problem. Given a finite collection of handlers and constraints describing their relative precedence, the compiler can compute a linear extension of those constraints and generate the corresponding nested `handle` expression.

This approach is compatible with the structural restriction of the arrow calculus. In the semantic setting considered by Sanada, the algebraic structure associated with arrows corresponds to a sequence of operations rather than the branching operation trees associated with ordinary monads. In particular, the problematic case is runtime branching between effectful commands. Our synthesis pass does not introduce such branching: its output is a statically determined sequence of nested handlers.

The contributions of this paper are:

1. **A declarative handler-topology model.** We define handler declarations independently of their syntactic nesting and express precedence as a relation over handlers.
2. **A static synthesis algorithm.** We reduce handler-topology synthesis to finding a linear extension of the declared precedence relation and lower the result to nested `handle` expressions.
3. **Structural correctness properties.** We establish that synthesis produces valid arrow-calculus terms and preserves declared handler precedence.
4. **A minimal implementation.** We describe a compile-time AST transformation for arrow-effect languages that reuses existing handler representations and virtual machines.
5. **An explicit expressiveness boundary.** We distinguish static synthesis, which produces a fixed handler topology, from runtime dynamic selection of effectful computations, which lies outside the arrow semantics considered here.
6. **A Lean 4 formalization.** The correctness properties are mechanically verified in Lean 4 with no admitted proofs (§7).

2. Background

2.1 Sanada's Arrow Calculus

Sanada extends the arrow calculus of Lindley, Wadler, and Yallop with operations and handlers.

The relevant syntax (Figure 2 of the thesis) is:

```
Terms       M, N, L ::= x |  M, N  | fst M | snd M |  x:A. M | M N |  •x:A. P
Values      V, W    ::= x |  V, W  |  x:A. M |  •x:A. P
Commands    P, Q, R ::=  M  | let x   P in Q | L • M | op(M) | handle R with H
Handlers    H       ::= { x:C → P }   { op, k: _D z:  → Q_op }_{op   Σ}
```

A handler `H` consists of a return clause `{ x:C → P }` and a set of operation clauses `{ op, k: _D z:  → Q_op }`. Each operation clause binds:

- `z :` — the operation argument (arity);
- `k :  _D` — the continuation (coarity);
- `Q_op` — the handler body for that operation.

The construct

```
handle R with H
```

evaluates command `R` under handler `H`.

Handler nesting determines which handler is considered first when an operation is encountered. Thus the following two terms are structurally different:

```
handle (handle P with H ) with H
```

and:

```
handle (handle P with H ) with H
```

A declarative system must therefore synthesize this ordering explicitly.

2.2 Handler Topology

We distinguish handler topology from computational topology.

Handler topology is the nesting relation between effect handlers:

```
H
   H
        H
             P
```

Computational topology, by contrast, describes dependencies between computations.

The present paper concerns only the former.

This distinction is important because the system does not infer a dataflow graph or physical layout. Its output is a valid handler nesting that realizes a declaratively specified precedence relation.

2.3 Arrow Operations and Sequential Structure

Sanada's semantic treatment of arrow effects is based on strong promonads. A relevant consequence is that the algebraic structure associated with arrows represents a sequence of operations rather than the branching operation trees associated with ordinary monadic effects.

This restriction has two consequences for our system.

First, a declarative handler topology must ultimately produce a linear nesting of handlers.

Second, a runtime construct that dynamically chooses between effectful commands based on the result of another effect cannot simply be introduced into the arrow calculus without changing its semantic model.

Our synthesis mechanism avoids both issues by computing the handler ordering statically.

3. The Declarative Handler Model

3.1 Handler Declarations

Let a declarative handler specification be:

```
D = (H,  )
```

where:

- `H` is a finite set of handler declarations;
- is a precedence relation over `H`.

For example:

```
fetch   cache
cache   log
```

states that `fetch` must be nested inside `cache`, and `cache` must be nested inside `log`.

The relation is not itself a nesting tree. It describes constraints on the eventual linear handler topology.

3.2 Partial Orders

The precedence relation is assumed to form a partial order.

Thus it must be:

- irreflexive;
- transitive;
- asymmetric.

If two handlers are unrelated by the declared relation, the compiler is free to choose either relative order, subject to any additional semantic requirements imposed by the implementation.

If the relation contains a cycle:

```
H   H
H   H
H   H
```

then no valid linear handler topology exists.

The compiler must reject such a specification rather than arbitrarily choosing an order.

3.3 Linear Extensions

A **linear extension** of `(H, )` is a sequence:

```
H , H , ..., H
```

such that:

```
H   H   i < j
```

The synthesis problem is therefore:

> Given a finite declarative handler specification `(H, )`, compute a linear extension of its precedence relation.

A standard topological-sort algorithm is sufficient for this purpose.

The important point is that this is a compile-time ordering problem. No runtime branching is necessary.

4. Handler Topology Synthesis

4.1 Synthesis Function

Let:

```
S(D, P)
```

be the synthesis function that takes a declarative specification `D` and an arrow command `P`.

Suppose the linear extension computed from `D` is:

```
H , H , ..., H
```

The synthesized command is:

```
handle (... handle (handle P with H ) with H  ... ) with H
```

Equivalently:

```
S(D, P) =
  handle (...(handle (handle P with H ) with H )... ) with H
```

Thus the first handler in the linear extension is the innermost handler.

4.2 Example

Consider:

```
events:

  - handler: FetchHandler
    handles: [fetch]
    priority: 10

  - handler: CacheHandler
    handles: [cache]
    priority: 20

  - handler: LogHandler
    handles: [log]
    priority: 30

body: ...
```

The priority declarations induce:

```
FetchHandler   CacheHandler   LogHandler
```

The synthesizer produces:

```
handle (
  handle (
    handle body with FetchHandler
  ) with CacheHandler
) with LogHandler
```

The declarative representation therefore describes the ordering constraint without requiring the programmer to manually encode the nesting.

4.3 Priority as a Convenience Syntax

A total numeric priority is one convenient way to specify a precedence relation:

```
priority(H ) < priority(H )
```

implies:

```
H   H
```

However, priority values are not fundamental to the model.

The more general representation is a partial-order specification:

```
H   H
H   H
```

which leaves `H` and `H` unordered.

This distinction allows the compiler to reject genuinely contradictory constraints while avoiding unnecessary ordering requirements.

5. Synthesis Algorithm

5.1 Algorithm

Given:

```
D = (H, )
```

and body `P`:

1. Validate that is acyclic.
2. Compute a linear extension of .
3. Initialize `R = P`.
4. For each handler in the linear extension, from innermost to outermost:
   - construct a `handle` node containing `R`;
   - replace `R` with the new node.
5. Return `R`.

In pseudocode:

```
synthesize(D, P):
    ordered = topological_sort(D.precedence)

    result = P

    for H in ordered:
        result = handle(result, H)

    return result
```

5.2 Complexity

Let:

- `n = |H|` be the number of handlers;
- `m = | |` be the number of precedence constraints.

Using a standard topological-sort algorithm, ordering requires:

```
O(n + m)
```

time.

Constructing the resulting nested AST requires:

```
O(n)
```

additional time.

Thus the synthesis phase is linear in the size of the handler specification, assuming the precedence relation is represented explicitly.

5.3 Determinism and Multiple Linear Extensions

A partial order may have multiple valid linear extensions. For example, given:

```
A   C
B   C
```

both `[A, B, C]` and `[B, A, C]` are valid linear extensions. The choice between them affects which of `A` or `B` is the innermost handler, which may in turn affect runtime behavior if `A` and `B` intercept overlapping operations.

For reproducible builds, the compiler can impose a deterministic tie-breaking rule, such as declaration order or handler identifier.

This does not change the declared precedence relation:

```
H   H
```

remains mandatory, while unrelated handlers receive a deterministic but otherwise arbitrary ordering.

The question of when two handlers can be safely reordered without changing behavior is a semantic question (see §14.1).

6. Correctness

The central correctness properties concern the relationship between the declarative specification and the generated syntax.

6.1 Well-Formedness

We define a command as **well-formed** if every handler in it has at least one operation clause and all sub-terms are well-formed. This is a structural property checked recursively:

```
isWellFormed(handle P with H) = |H.clauses| > 0   isWellFormed(P)
isWellFormed(let x   P in Q)   = isWellFormed(P)   isWellFormed(Q)
```

```
isWellFormed(cond P Q R)       = isWellFormed(P)   isWellFormed(Q)   isWellFormed(R)
isWellFormed(op(M))            = true
isWellFormed( M )              = true
```

Proposition 1 — Well-Formedness.

For every finite acyclic handler specification `D = (H, )` and well-formed command `P`, if every handler in the linear extension of has at least one clause, then `S(D, P)` is well-formed.

Proof sketch.

The synthesis folds `handle` over the linear extension. Each step wraps the current term with a handler `H` that has at least one clause. By the definition of `isWellFormed`, wrapping a well-formed term with a handler that has clauses preserves well-formedness. Induction over the handler sequence establishes the result. The accompanying Lean formalization proves this as `synthesize_well_formed`.

6.2 Precedence Preservation

Proposition 2 — Precedence Preservation.

For every pair of handlers `H` and `H` :

```
H   H
```

implies that `H` is nested inside `H` in `S(D, P)`.

Proof sketch.

The synthesis algorithm uses a linear extension of . By definition of a linear extension:

```
H   H   position(H ) < position(H )
```

Since handlers are folded from the innermost position toward the outermost position, `H` is constructed before `H` and is therefore nested inside it.

The key invariant is that if `H` appears before `H` in the ordered list, then for any body term, `H` is nested inside `H` in the result of folding `handle` over that list. This is proved by induction on the `AppearsBefore` relation, with two helper lemmas handling the fold accumulation pattern. The accompanying Lean formalization proves this as `synthesize_preserves_precedence`.

Corollary — Completeness.

If `ordered` is a linear extension of (i.e., `a b, a  b  AppearsBefore(a, b, ordered)`), then for every constraint `H  H` in , `H` is nested inside `H` in `synthesize(P, ordered)`.

This follows directly from Proposition 2: the linear extension property ensures `AppearsBefore(H , H , ordered)`, and Proposition 2 converts this into structural nesting. The Lean formalization proves this as `synthesize_completeness`.

The soundness of the topological sort — the fact that it produces a list satisfying the linear extension property — is also formally verified, as `exists_linear_extension` in `TopoSort.lean` (§7.2). This theorem proves that for any finite list of handlers with an irreflexive and transitive precedence relation, there exists a linear extension realizing all precedence constraints as `AppearsBefore`.

6.3 No Introduced Runtime Branching

We define a command as **static** if it contains no `cond` (runtime conditional branching) constructor. The `isStatic` predicate is checked recursively:

```
isStatic(handle P with H) = isStatic(P)
isStatic(cond P Q R)      = false
isStatic(let x   P in Q)  = isStatic(P)   isStatic(Q)
isStatic(op(M))           = true
isStatic( M )             = true
```

Proposition 3 — Static Topology.

If the body command `P` is static (contains no `cond`), then `S(D, P)` is also static. Synthesis introduces no runtime branching over effectful commands.

Proof sketch.

The synthesis function folds `handle` over the ordered list. Since `isStatic(handle P with H) = isStatic(P)`, wrapping with a handler preserves staticity. By induction over the handler list, if the initial body is static, the result remains static. The `cond` constructor is never introduced by synthesis — it only appears if it was already present in the body.

The accompanying Lean formalization proves this as `synthesize_static_topology`, with the `cond` constructor explicitly included in the `Command` type to make the theorem non-vacuous.

6.4 Semantic Boundary

The preceding property should not be interpreted as proving that arbitrary dynamic effect-routing programs can be represented in the arrow calculus.

Rather, the result establishes a narrower fact:

> A static declarative description of handler precedence can be compiled into the existing static handler-nesting mechanism without introducing the dynamic command branching excluded by the underlying arrow model.

This distinction is central to the scope of the system.

7. Formalization in Lean 4

The correctness properties of §6 have been mechanically verified in Lean 4. The formalization is split across the following files under `Dts/`:

- `Basic.lean` — core types (`Command`, `Handler`, `isStatic`, `isWellFormed`)
- `Precedence.lean` — precedence model, partial orders, acyclicity
- `Synthesis.lean` — synthesis function, well-formedness, precedence preservation, staticity, completeness
- `TopoSort.lean` — topological sort soundness (`exists_linear_extension`)
- `Semantics.lean` — trace semantics for `Command` (observable handler-dispatch events)
- `SemanticEquivalence.lean` — semantic equivalence of synthesized and manually nested terms
- `Commutativity.lean` — semantic commutativity of disjoint handlers, with counterexample

The root `Dts.lean` imports all modules. The project compiles with no `sorry` (no admitted proofs). Mathlib is used for tactic utilities only (e.g., `omega`, `push_neg`); no mathematical content is imported from it.

7.1 Model

The Lean model mirrors the paper's definitions:

- `Command` is an inductive type with constructors `pure`, `op`, `letin`, `handle`, and `cond`. The `cond` constructor represents runtime conditional branching; its inclusion makes the staticity theorem (Proposition 3) non-vacuous.
- `Precedence` is a function `Handler → Handler → Bool`.
- `synthesize body ordered` is a left fold over the ordered handler list, matching the paper's `S(D, P)`.
- `AppearsBefore` is an inductive predicate capturing list ordering without relying on numeric indices.
- `NestedInside` is an inductive predicate capturing handler nesting structure.

7.2 Proven Theorems

- `irreflexive_transitive_implies_asymmetric` — Lemma: asymmetry is derivable (not assumed)
- `partialOrder_acyclic` — Acyclicity of partial orders
- `synthesize_well_formed` — Proposition 1 (conditional on non-empty clauses)
- `synthesize_preserves_precedence` — Proposition 2
- `synthesize_static_topology` — Proposition 3 (conditional on body being static)
- `synthesize_completeness` — Corollary (completeness given linear extension)
- `reachable_implies_edge` — Lemma: reachability implies a direct edge (used by acyclicity)
- `exists_linear_extension` — Topological sort soundness (existence of linear extension)
- `synthesize_eq_manuallyNested` — Syntactic equality of synthesis and explicit nesting
- `synthesize_equiv_manuallyNested` — Semantic equivalence (Corollary: equal syntax equal traces)
- `synthesize_commutative_two` — Semantic commutativity for disjoint handlers (§14.1)

7.3 Assumptions

The formalization makes no unproven assumptions. The linear extension property used by `synthesize_completeness` is established by `exists_linear_extension` in `TopoSort.lean`, which proves that for any finite list of handlers with an irreflexive and transitive precedence relation, a valid linear extension exists. This is a constructive version of a standard result in order theory (Szpilrajn's extension theorem).

8. Relationship to Arrow Expressiveness

8.1 Dynamic Command Branching

Consider a hypothetical program:

```
v   op()
if v then P  else P
```

where `P` and `P` are effectful commands.

The branch selects a future effectful computation based on the result of an operation. This is fundamentally different from static handler synthesis.

Our system instead performs:

```
compile time:
    determine H   H   H

execution time:
    execute fixed nesting
```

The handler topology is therefore independent of runtime operation results.

8.2 Pure Term-Level Branching

The restriction does not imply that all conditional behavior disappears.

A pure value can still participate in ordinary term-level computation. The relevant distinction is whether the branch dynamically selects between effectful command structures.

Our compiler does not need to alter this distinction.

8.3 Why the Restriction Is Useful

The same restriction that limits dynamic expressiveness also simplifies static synthesis.

If arbitrary runtime command branching were permitted, the handler topology might itself depend on runtime execution paths. A compiler would then potentially need to synthesize a branching handler structure.

Under the arrow model considered here, static handler topology remains a linear structure:

```
H
 ↓
H
 ↓
H
 ↓
P
```

This makes compile-time synthesis particularly simple.

9. Declarative Language Design

9.1 Syntax

A concrete configuration syntax can expose handler precedence without exposing the underlying nesting.

For example:

```
events:
  - handler: Cache
    before: [Logging]

  - handler: Authentication
    before: [Cache]

  - handler: Logging

body:
  application
```

This corresponds to:

```
Authentication  Cache  Logging
```

The configuration language need not expose `handle` directly.

9.2 Explicit Relations versus Priorities

We recommend treating explicit precedence constraints as the semantic core:

```
Authentication before Cache
Cache before Logging
```

Numeric priorities can be provided as syntactic sugar:

```
Authentication: 10
Cache: 20
Logging: 30
```

This avoids coupling the language semantics to an arbitrary numeric scale.

9.3 Invalid Specifications

The compiler should diagnose at least:

- cyclic precedence constraints;
- duplicate handler identities;
- references to undefined handlers;
- malformed handler clauses;
- conflicting declarations.

For example:

```
A before B
B before A
```

must produce a compile-time error because no valid linear nesting satisfies both constraints.

10. Implementation

The proposed implementation is deliberately small.

The declarative frontend produces handler declarations and precedence constraints. A synthesis pass then transforms the existing AST before VM evaluation.

Conceptually:

```
Declarative specification


 Handler declarations


 Precedence graph


 Topological sort


 Nested handler expressions
```

```
Existing compiler / VM
```

The synthesis pass is therefore a pure AST transformation.

A simplified implementation in pseudocode:

```
synthesize_topology(body, handlers):
    ordered = topological_sort(handlers.precedence)

    result = body

    for H in ordered:
        result = handle(result, H)

    return result
```

The implementation does not require a new runtime representation of handlers. The generated nodes use the existing handler-expression representation of the target language.

Likewise, the VM continues to perform ordinary handler lookup and evaluation.

The principal implementation advantage is therefore not that handlers become intrinsically cheaper, but that the declarative layer introduces **no additional runtime dispatch mechanism**.

11. Evaluation Plan

11.1 Correctness of Ordering

Generate handler specifications with known precedence relations and verify that the resulting AST has the corresponding nesting.

For example:

```
A   B   C
```

must produce:

```
handle(
  handle(
    handle(P, A),
    B
  ),
  C
)
```

11.2 Detection of Invalid Topologies

Generate cyclic specifications and verify that synthesis rejects them at compile time.

Examples include:

```
A  B
B  A
```

and:

```
A  B
B  C
C  A
```

11.3 Behavioral Equivalence

For specifications whose nesting is known manually, compare:

```
manually nested program
```

with:

```
declaratively synthesized program
```

and verify equivalent behavior under the existing VM.

This experiment is more significant than simply checking the AST shape because handler nesting can affect which operation clause receives an operation.

11.4 Compilation Cost

Measure synthesis time as the number of handlers and precedence constraints increases.

The expected complexity is:

```
O(|H| + | |)
```

for the ordering phase.

The experiment should verify that synthesis remains negligible relative to ordinary compilation and execution costs for realistic handler counts.

12. Relationship to Prior Work

12.1 The Arrow Calculus

The arrow calculus of Lindley, Wadler, and Yallop provides the foundational calculus for reasoning about computations with explicit structure.

Our work does not introduce a new arrow calculus. Instead, it uses the existing handler mechanism as the target of a declarative compilation layer.

12.2 Lindley's _flow

Lindley's _flow explored algebraic effects and handlers across idioms, arrows, and monads and provided static effect distinctions.

This work is relevant because it demonstrates that effect structure can be exposed statically.

Our contribution is different: we use a declarative specification to **construct the handler topology itself**.

Thus the relationship can be summarized as:

```
 _flow

    static effect structure

Sanada

    formal arrow handlers

Declarative synthesis

    handler precedence

nested handle expressions
```

12.3 Algebraic Effects and Handlers

General algebraic-effects research establishes the separation between effect declaration and interpretation.

Our system applies this principle specifically to handler composition: the declarative specification describes how handlers should relate, while the compiler determines the concrete nesting required by the target calculus.

13. Limitations

The system intentionally has a narrow scope.

First, it synthesizes a **static linear handler topology**. It does not synthesize arbitrary runtime handler graphs.

Second, the current model does not attempt to infer handler precedence automatically from arbitrary program behavior. Precedence must either be declared or derived from a separately specified static policy.

Third, a partial order may admit multiple valid linear extensions. Although deterministic tie-breaking can provide reproducible output, different valid extensions may have different runtime

behavior when handlers interact. A complete system therefore needs a semantic account of when two handlers may be safely reordered.

Fourth, the present work does not establish that every useful effect system can be represented by the arrow model. Its claims are restricted to handler topology that can be represented by the existing arrow calculus.

Finally, the current implementation does not optimize handler composition. Its purpose is to establish the correctness and practicality of the declarative-to-nested transformation.

14. Future Work

14.1 Semantic Commutativity

An important question is when two handlers can safely be reordered.

We provide a partial answer, formalized in `Commutativity.lean`. Two handlers are **clause-disjoint** when no operation name is handled by both:

```
disjointClauses(H , H ) :=  op. ¬(H  handles op  H  handles op)
```

Theorem 4 — Two-Handler Commutativity. For a static body `P`, if `H` and `H` are clause-disjoint, then:

```
eval(synthesize(P, [H , H ])) = eval(synthesize(P, [H , H ]))
```

That is, the trace is identical regardless of which disjoint handler is innermost.

The proof proceeds by showing that swapping two disjoint adjacent handlers in the handler context does not change which handler first catches any operation (`firstHandler_swap`), and then lifting this to all static command forms by induction (`evalWith_swap`).

A counterexample demonstrates that the disjointness condition is necessary: when two handlers both handle the same operation, the innermost handler catches it, so reordering changes the trace.

The general case — arbitrary permutations of pairwise-disjoint handler lists — is left as future work. The two-handler case is fully proved.

14.2 Inferred Precedence

Some precedence constraints could potentially be inferred from operation signatures or handler dependencies.

For example, if a handler invokes an operation that another handler must intercept, the compiler could derive a necessary nesting relationship.

14.3 Optimization

A synthesis pass could eliminate redundant handlers, merge compatible handlers, or specialize handler nesting when their operation sets are statically known.

14.4 Beyond Formalization

The Lean formalization (§7) establishes both structural and semantic correctness of synthesis: well-formedness, precedence preservation, staticity, completeness, existence of linear extensions, semantic equivalence between synthesized and manually nested terms (§7.2), and two-handler commutativity for disjoint handlers (§14.1). Further formalization work could verify:

1. The general commutativity conjecture: for a static body, if all handlers in a list are pairwise clause-disjoint, then any permutation of that list produces the same trace.
2. Semantic equivalence under a richer operational semantics that models continuation capture and resumption, beyond the trace semantics currently formalized.

14.5 Beyond Total Handler Order

Although the final arrow representation is linear, future work could investigate richer declarative descriptions that compile into multiple statically selected handler configurations. For example, a specification might describe two alternative handler orderings guarded by a compile-time condition (e.g. debug vs production mode). The compiler would then generate two separate nested `handle` expressions and select between them using permitted term-level branching (`if M then N else N` where `N` and `N` are terms, not commands). This stays within the arrow calculus because the selection is static or term-level — it does not introduce runtime branching on effectful commands. The key insight is that multiple static topologies can coexist as long as the choice between them does not depend on operation results.

15. Conclusion

We presented a declarative system for synthesizing handler topology in an arrow calculus with algebraic effects and handlers.

The key idea is to separate **handler specification** from **handler syntax**. A programmer declares a set of handlers and a precedence relation. The compiler computes a linear extension of that relation and lowers it to the nested `handle` expressions required by the underlying calculus.

This transformation is fundamentally static. It does not introduce runtime branching over effectful commands and therefore does not require an extension of the arrow semantic model. The synthesis problem is consequently reduced to a well-defined compile-time ordering problem. Its correctness has been mechanically verified in Lean 4 (§7), covering well-formedness, precedence preservation, staticity, completeness, semantic equivalence, and two-handler commutativity for disjoint handlers.

The approach also provides a minimal implementation path. In a language with existing handler-expression AST nodes, the declarative system can be implemented as an AST-to-AST transformation producing those nodes. The existing runtime remains responsible for ordinary handler evaluation.

The principal result is therefore not a new runtime effect mechanism, but a new **declarative layer for constructing an existing arrow-effect mechanism**. The boundary is precise:

```
static handler topology
        + synthesizable

runtime effect-dependent command topology
        - not introduced by this system
```

This boundary turns a limitation of arrow effects into a useful design constraint. Because the generated handler structure is linear and statically known, its synthesis is simple, deterministic, and amenable to formal verification.

The resulting system provides a small example of a broader principle: when a programming language exposes a structurally constrained semantic representation, a declarative layer can synthesize that representation without changing the underlying runtime. In the present case, Sanada's explicit handler topology becomes not merely a syntactic requirement, but the target of a compile-time synthesis process.

References

1. Sanada, T. (2024). *Algebraic effects and handlers for arrows.* Journal of Functional Programming, 34. doi:10.1017/S0956796824000066
2. Sanada, T. (2024). *Algebraic effects and handlers for arrows.* PhD thesis, Kyoto University. https://www.kurims.kyoto-u.ac.jp/~tsanada/papers/phdthesis.pdf
3. Lindley, S. (2014). *Algebraic effects and effect handlers for idioms and arrows.* In Proceedings of the 19th International Conference on Functional Programming (ICFP). doi:10.1145/2633628.2633636
4. Lindley, S., Wadler, P., and Yallop, J. (2010). *The arrow calculus.* Journal of Functional Programming, 20(1), pp. 51–87.
5. Plotkin, G. and Pretnar, M. (2013). *Handling algebraic effects.* Logical Methods in Computer Science, 9(4).
6. Hughes, J. (2000). *Generalising monads to arrows.* Science of Computer Programming, 37(1–3), pp. 67–111.